

Document number: P0357R0
Date: 2016-05-24
Project: Programming Language C++, Library Evolution Working Group
Reply-to: Tomasz Kamiński <tomaszkam at gmail dot com>
Stephan T. Lavavej <stl at microsoft.com>

reference_wrapper for incomplete types

1. Introduction

This paper proposes change in `reference_wrapper` specification to support incomplete types.

In addition to necessary removal of `result_type` and related typedefs for `reference_wrapper`, the paper proposes to remove all deprecated functional components placed in D7. Old adaptable function bindings [depr.func.adaptor.binding] section of the standard. As consequence this paper may be viewed as successor of [P0005R4: Adopt not_fn from Library Fundamentals 2 for C++17](#) paper.

2. Motivation and Scope

`std::reference_wrapper` is an utility class created to allow to use reference in the interface that was designed to pass objects by value. However design of the standard component has a major drawback, when compared to the alternative solutions based on the use of the raw pointers of boost version of this component: the referenced type is required to be complete. As consequence, depending on the context use of `reference_wrapper` may increase compilation time by adding a new definition, or even be impossible in case when definition of the class is not available to the programmer.

Moreover `std::reference_wrapper` specializations are recognized by standard factory function, like: `std::make_pair`, `std::make_tuple` or `std::bind` and allows the programmer to create tuple of references by use of:

```
std::make_tuple(std::ref(t), std::ref(u))
```

Use of this feature, not only avoids cumbersome specification of the type, but also eliminates the possibility of encountering dangling reference problems that it may introduce. For example in case of following declaration:

```
std::pair<std::string const&, int> p("test", 10);
```

every use of `p.first` leads to undefined behaviour caused by read of dangling reference. Despite of all of above advantages, programmes are still forced to use `pair<T&, U&>` solution, when at least one of types `T` or `U` is incomplete.

3. Design Decisions

Supporting incomplete types in `reference_wrapper` is currently impossible because the implementation is required to check the template parameter for presence of `result_type` and related nested types. As consequence support for this (now deprecated) features needs to be removed from standard.

3.1. Removal of deprecated functional components

Despite the fact that support for incomplete types require only removal support for *weak result type* and argument typedefs for `reference_wrapper`, this paper follow the original direction of [P005R4: Adopt not_fn from Library Fundamentals 2 for C++17](#) and proposes removal of all deprecated function bindings. As this paper is targeted for next standard after C++17, it also follows committee guideline to introduce period of deprecation before removal.

In addition feature proposed with the paper conflicts with support for the old function binding protocol and vendors will not longer be allowed to provide it in their `std::reference_wrapper` implementations.

3.2. Support for `is_transparent`

In the C++14, another protocol based on the presence of the `is_transparent` nested type was introduced, to indicate that given functor enables heterogeneous lookup for associative container. As in case of `result_type` implementation of this protocol in exact form for `reference_wrapper<T>` would reintroduce requirement of completeness of `T` template parameter.

Despite the fact that support for incomplete types and heterogeneous container lookup in `reference_wrapper` may look incompatible, there is possibility to provide both of them, via alternative design that rely on use metafunction instead of nested

type, as proposed in [P0046R1: Change is transparent to metafunction \(Revision 1\)](#).

4. Impact On The Standard

This proposal depends on the deprecation of the `result_type` and related typedefs in the C++17, so they can be removed in upcoming standard after period of deprecation.

Nothing depends on this proposal.

5. Proposed Wording

The proposed wording changes refer to [N4582](#) (C++ Working Draft, 2016-03-19).

Apply following changes to paragraph 17.6.4.3.1 Zombie names [zombie.names]:

In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, [not1](#), [not2](#), [unary_negate](#), [binary_negate](#), `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, and `unary_function`.

[It is unspecified whether function objects in the C++ standard library additionally provide the following typedefs: `result\_type`, `argument\_type`, `first\_argument\_type`, and `second\_argument\_type`.](#)

At the end of the section 20.12.4 Class template `reference_wrapper` [refwrap]:

[The template parameter `T` of `reference\_wrapper` may be an incomplete type.](#)

Apply following changes to paragraph 20.12.4.4 `reference_wrapper` invocation [refwrap.invoke]:

```
template <class... ArgTypes>
    result_of_t<T&(ArgTypes&&... )>
        operator()(ArgTypes&&... args) const;
```

Returns: `INVOKE(get(), std::forward<ArgTypes>(args)...) .` ([func.require] 20.12.2).

[Remarks: If `T` is an incomplete type, the program is ill-formed.](#)

At the begining of the paragraph 20.12.4.5 `reference_wrapper` helper functions [refwrap.helpers]:

[The template parameter `T` of the following `ref` and `cref` function templates may be an incomplete type.](#)

Under Annex C Compatibility [diff] add new subclause after C.4 C++ and ISO C++ 2014 [diff.cpp14]:

C.x C++ and ISO C++ 2017

[diff.cpp17]

[This subclause lists the differences between C++ and ISO C++ 2017 \(TBD\), by the chapters of this document.](#)

C.x.1 Clause 20: general utilities library

[diff.cpp17.utilities]

Change: [The typedefs `result\_type`, `argument\_type`, `first\_argument\_type`, and `second\_argument\_type` might not be defined by some function objects.](#)

Rationale: [Superseded by new features, including `decltype` and forwarding references.](#)

Effect on original feature: [Valid C++ 2017 code that uses these typedefs may fail to compile in this International Standard.](#)

Change: The class templates `unary_negate` and `binary_negate` and the function templates `not1` and `not2` might not be defined.

Rationale: Superseded by new features, including generic lambdas and the function template `not_fn`.

Effect on original feature: Valid C++ 2017 code that uses these class templates and function templates may fail to compile in this International Standard.

Remove clause D7. Old adaptable function bindings [depr.func.adaptor.binding] entirely.

6. Implementability

Without requirement to conditionally support `result_type` and related typedefs, straightforward implementation provides support for incomplete types.

```
template<typename T>
class reference_wrapper
{
    T* ptr;

public:
    using type = T;

    reference_wrapper(T& val) noexcept
        : ptr(std::addressof(val))
    {}

    reference_wrapper(T&&) = delete;

    T& get() const noexcept { return *ptr; }
    operator T&() const noexcept { return *ptr; }

    template<typename... Args>
    auto operator()(Args&&... args)
        -> std::result_of_t<T&(Args...)>
    { return std::invoke(*ptr, std::forward<Args>(args)...); }
};
```

7. Acknowledgements

Special thanks and recognition goes to Sabre (<http://www.sabre.com>) for supporting the production of this proposal, and for sponsoring Tomasz Kamiński's trip to the Oulu for WG21 meeting.

8. References

1. Alisdair Meredith, Stephan T. Lavavej, Tomasz Kamiński, "Adopt `not_fn` from Library Fundamentals 2 for C++17", (P0005R4, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0005r4.html>)
2. Tomasz Kamiński, "Change `is_transparent` to metafunction (Revision 1)", (P0046R1, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0046r1.html>)
3. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4582, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4582.pdf>)